

LAB ACTIVITY (Exp-08)

Q1) Represent tree data structure with array

```
#include <stdio.h>
#define MAX 50

int tree[MAX];
int n;

void inputTree() {
    printf("Enter number of nodes in the binary tree: ");
    scanf("%d", &n);

    printf("Enter %d node values (level-wise):\n", n);
    for (int i = 0; i < n; i++) {
        printf("Node %d: ", i);
        scanf("%d", &tree[i]);
    }
}

void displayTree() {
    printf("\nBinary Tree (Array Representation):\n");
    for (int i = 0; i < n; i++) {
        printf("Index %d-->Data:- %d\n", i, tree[i]);
    }
}

void showRelations() {
    printf("\nParent-Child Relationships:\n");
    for (int i = 0; i < n; i++) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        printf("Parent: %d ", tree[i]);
        if (left < n)
            printf("| Left Child: %d", tree[left]);
        else
            printf("| Left Child: None");

        if (right < n)
            printf(" | Right Child: %d", tree[right]);
        else
            printf(" | Right Child: None");

        printf("\n");
    }
}

int main() {
    inputTree();
    displayTree();
    showRelations();
    return 0;
}
```

```
Enter number of nodes in the binary tree: 6
```

```
Enter 6 node values (level-wise):
```

```
Node 0: 0
```

```
Node 1: 1
```

```
Node 2: 2
```

```
Node 3: 3
```

```
Node 4: 4
```

```
Node 5: 5
```

```
Binary Tree (Array Representation):
```

```
Index 0-->Data:- 0
```

```
Index 1-->Data:- 1
```

```
Index 2-->Data:- 2
```

```
Index 3-->Data:- 3
```

```
Index 4-->Data:- 4
```

```
Index 5-->Data:- 5
```

```
Parent-Child Relationships:
```

```
Parent: 0 | Left Child: 1 | Right Child: 2
```

```
Parent: 1 | Left Child: 3 | Right Child: 4
```

```
Parent: 2 | Left Child: 5 | Right Child: None
```

```
Parent: 3 | Left Child: None | Right Child: None
```

```
Parent: 4 | Left Child: None | Right Child: None
```

```
Parent: 5 | Left Child: None | Right Child: None
```

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab9(1510)> █
```

Q2) Represent Tree data structure with linked list

Q3) apply pre, in, post, level order traversal to display

Q4) search an element from a binary tree

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* createTree() {
```

```

int data;
printf("Enter root node data (-1 for no tree): ");
scanf("%d", &data);

if (data == -1)
    return NULL;

struct Node* root = createNode(data);

struct Node* queue[50];
int front = 0, rear = 0;
queue[rear++] = root;

while (front < rear) {
    struct Node* current = queue[front++];

    printf("Enter left child of %d (-1 for no left child): ", current->data);
    scanf("%d", &data);
    if (data != -1) {
        current->left = createNode(data);
        queue[rear++] = current->left;
    }

    printf("Enter right child of %d (-1 for no right child): ", current->data);
    scanf("%d", &data);
    if (data != -1) {
        current->right = createNode(data);
        queue[rear++] = current->right;
    }
}

return root;
}

void levelOrder(struct Node* root) {
    if (root == NULL) {
        printf("Tree is empty.\n");
        return;
    }

    struct Node* queue[50];
    int front = 0, rear = 0;

    queue[rear++] = root;

    printf("Level Order Traversal: ");
    while (front < rear) {
        struct Node* current = queue[front++];
        printf("%d ", current->data);

        if (current->left)
            queue[rear++] = current->left;
        if (current->right)
            queue[rear++] = current->right;
    }
}

void preorder(struct Node* root) {
    if (root == NULL) return;

```

```

    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}

void inorder(struct Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

void postorder(struct Node* root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}

int search(struct Node* root, int key) {
    if (root == NULL)
        return 0;

    if (root->data == key)
        return 1;

    return search(root->left, key) || search(root->right, key);
}

int main() {
    struct Node* root = createTree();
    printf("\n");
    levelOrder(root);

    printf("\nPreorder Traversal: ");
    preorder(root);

    printf("\nInorder Traversal: ");
    inorder(root);

    printf("\nPostorder Traversal: ");
    postorder(root);

    int key;
    printf("\n\nEnter element to search: ");
    scanf("%d", &key);

    if (search(root, key))
        printf("Element %d found in the tree.\n", key);
    else
        printf("Element %d not found in the tree.\n", key);

    return 0;
}

```

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab9(1510)> cd "d:\SEMESTER I\LAB WORK\Data structures and algorithm\lab9(1510)"
($?) { gcc tree_linked.c -o tree_linked } ; if ($?) { .\tree_linked }
Enter root node data (-1 for no tree): 0
Enter left child of 0 (-1 for no left child): 1
Enter right child of 0 (-1 for no right child): 2
Enter left child of 1 (-1 for no left child): 3
Enter right child of 1 (-1 for no right child): 4
Enter left child of 2 (-1 for no left child): 5
Enter right child of 2 (-1 for no right child): 6
Enter left child of 3 (-1 for no left child): -1
Enter right child of 3 (-1 for no right child): -1
Enter left child of 4 (-1 for no left child): -1
Enter right child of 4 (-1 for no right child): -1
Enter left child of 5 (-1 for no left child): -1
Enter right child of 5 (-1 for no right child): -1
Enter left child of 6 (-1 for no left child): -1
Enter right child of 6 (-1 for no right child): -1

Level Order Traversal: 0 1 2 3 4 5 6
Preorder Traversal: 0 1 3 4 2 5 6
Inorder Traversal: 3 1 4 0 5 2 6
Postorder Traversal: 3 4 1 5 6 2 0

Enter element to search: 5
Element 5 found in the tree.
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab9(1510)> |
```